Database APIs | Side by side

Shared Python contract

store.create_collection(spec)
store.upsert(records)
store.search(vector, k=5, filter=where, space=space)
store.delete(ids)
store.close()
Native persistence and capabilities vary; read the adapter contract.

Qdrant Python / TypeScript

Python: create_collection, upsert, query_points, delete.
TypeScript: createCollection, upsert, query, delete.
Named vectors use an explicit name. Payload filters are native; the shared adapter also maps validated eligible IDs.

Weaviate Python / TypeScript

Both clients use collections, data insertion and near-vector queries, with language-specific naming. Supply vectors explicitly and disable automatic vectorization in these lessons. Verify insert batch errors and collection cleanup.

SQL and embedded APIs

pgvector: SQL INSERT/ON CONFLICT and distance operators with ORDER BY/LIMIT. SQLite-vec: vector virtual table plus SQL. Chroma: collection.upsert/query/delete. LanceDB: table.add/search/delete; confirm replacement semantics rather than assuming add is an upsert.

Milvus and broader systems

MilvusClient uses create_collection, insert/upsert, search and delete. Elasticsearch, Redis, MongoDB and Vespa have distinct schema, query and readiness boundaries. Use Module 16's pinned native examples before translating operations.

Check every boundary

Stable business IDs, vector dimension/model space, replacement, server-side filter behavior, readiness, delete visibility and ownership of close/drop. API names are a navigation aid, not copy-paste executable calls. See common/stores and B1 for tested code.